

NEXUS-OP · OPERATIONAL INTELLIGENCE PLATFORM

Implementation Plan & Architecture Analysis

Full codebase audit · Gap analysis · Phased build roadmap · Priority action plan

Generated: 14 May 2026

v1.0 · MVP Stage

15

Modules built

22

API routes

6

Phases planned

35%

MVP readiness

CRITICAL

Auth status

Executive Summary

Nexus-OP is an AI-powered Infrastructure Operations Platform designed to become the central nervous system of project execution for construction, EPC, and government infrastructure projects. This document presents a full codebase audit, gap analysis against the stated roadmap, and a prioritised implementation plan to evolve from the current MVP foundation into a production-grade operational intelligence platform.

The codebase demonstrates strong domain knowledge — the BOQ → Indent → PO → GRN → Inventory → Billing chain is correctly modelled. However critical security, structural, and data integrity gaps must be resolved before onboarding any real organisation. This plan addresses those gaps with a concrete, phased approach.

Table of Contents

01	Codebase Architecture Overview	3
02	Module-by-Module Status Audit	4
03	Data Flow Analysis	6
04	Critical Issues & Security Gaps	7
05	Roadmap Gap Analysis (Phases 1–6)	8
06	Phased Implementation Plan	9
07	Priority Action Plan — What to Build Next	13
08	File Change Tracker	15
09	Architecture Refactor Guide	17
10	Success Metrics & Completion Criteria	19

01 · Codebase Architecture Overview

Current Stack

The project is a React 19 single-page application with an Express.js backend deployed to Vercel as a serverless function. Supabase handles the PostgreSQL database and will eventually handle auth. The frontend uses Tailwind CSS, React Router v7, Recharts for visualisations, React-Leaflet for maps, and ReactFlow for the process diagram.

Current Directory Structure

```
-nexus-op-main/
  api/
    index.js      ← ALL 22 routes in one 620-line file
    supabase.js   ← Supabase client (anon key, no RLS)
  src/
    App.jsx       ← Router + TopHeader (role/project switcher)
    components/
      Sidebar.jsx ← Role-filtered nav (UI only, no enforcement)
      ORRMap.jsx  ← Hardcoded Hyderabad ORR coordinates
    context/
      ProjectContext.jsx ← Global project + workorders state
      RoleContext.jsx   ← useState("Admin") - not real auth
    pages/          ← 15 page modules (mixed axios/fetch)
  package.json     ← express + puppeteer mixed with React deps
```

Critical Architectural Red Flags

- !

No auth on any route

Every API endpoint — including PO approval and bill generation — is fully public. Anyone with your Vercel URL can approve POs or generate bills without logging in.

CRITICAL
- !

Server packages in frontend package.json

express, cors, and puppeteer are in dependencies (not devDependencies). Puppeteer alone is 300MB+ including Chromium. Vite attempts to bundle all of this.

CRITICAL
- !

Supabase anon key + no RLS

Without Row Level Security policies, the anon key gives public read/write access to every table, bypassing Express routes entirely. Data isolation relies on ?projectId= query params which are trivially spoofable.

CRITICAL
- #

Single 620-line monolithic API file

All 22 routes, business logic, and DB queries live in one file. No controllers, services, or middleware separation. Untestable in isolation.

HIGH



Mixed fetch patterns

HIGH

Dashboard/PO/Vendors use axios. MB/BOQ/GRN/Indent use native fetch. No central API client. Error handling is inconsistent across modules.



No loading states or optimistic updates

MEDIUM

Pages fetch on mount with no skeleton UI. State updates wait for full server round-trip then re-fetch everything. Stale data visible on slow connections.

02 · Module-by-Module Status Audit

Status Legend

REAL	Fully functional, real DB interaction	PARTIAL	Works but incomplete or read-only
STUB	UI exists, logic is fake (setTimeout)	MISSING	Not built at all

Authentication & RBAC

Role is useState("Admin"). Zero backend enforcement. All 22 API routes are fully public. RBAC o...

MISSING

Projects

Full CRUD with type-based module gating (construction vs generic). ProjectContext propagates gl...

REAL

Work Orders

Bound to projects and vendors. Passed into context for child modules. Missing: WO-level budget ...

REAL

Vendors

"Smart PDF Import" is a setTimeout that hardcodes "ABC Infra Pvt Ltd". "Bulk Excel Import" call...

STUB

Purchase Orders

Best-implemented module. Real state machine: Pending → Approved → Dispatched → Delivered. Backe...

REAL

Indent

Status workflow (Pending/Approved/Rejected) works. Linked to BOQ and Work Orders. Critical gap:...

REAL

GRN

Validates PO must be Dispatched before accepting. Auto-updates inventory (upsert). Creates acti...

REAL

Inventory

Auto-managed via GRN — correct approach. But read-only in UI. No manual adjustment, no consumpt...

PARTIAL

BOQ

Item codes, units, rates, estimated quantities. MB data pulled in to show executed vs estimated...

REAL

Measurement Book

L×W×D calculation works. Tied to BOQ and Work Orders. No approval status on entries — all flow ...

PARTIAL

RA Bills

Generates from MB measurements with correct TDS (2%) and retention (5%). Status stuck at "Draft...

PARTIAL

Milestones

Lists milestones from API. No project filter on the API endpoint (fetches all globally). Read-o...

PARTIAL

Dashboard

4 stat cards are real. S-curve uses Math.random() — re-randomises on every render. ORR map has ...

STUB

Process Flow

ReactFlow + dagre auto-layout works. Visually impressive. Hardcoded nodes — not driven by live ...

PARTIAL

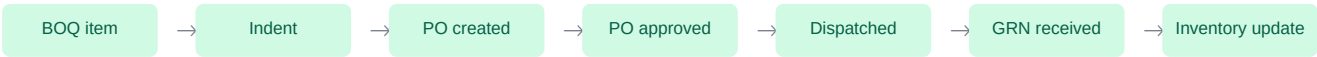
Activity Log

Only GRN creates activity entries. All other 21 routes (PO approvals, Indent changes, Bill gene...

STUB

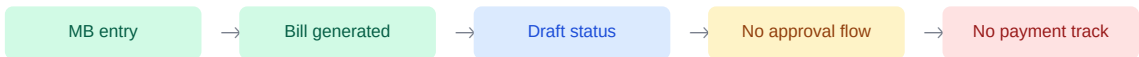
03 · Data Flow Analysis

The ONE real end-to-end flow — Procurement → Inventory



This flow is fully connected and correctly enforces state transitions. GRN validates Dispatched status, upserts inventory, marks PO Delivered, and creates an activity entry. This is the gold standard pattern to replicate across all other modules.

Billing flow — Works but terminates early



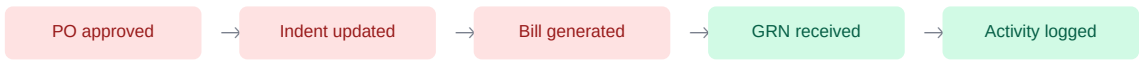
Bill generation correctly applies TDS and retention deductions. But bills never leave "Draft" — there is no submit → approve → pay lifecycle. The Indent → PO linkage (indentId field) exists in the schema but is not enforced or surfaced in the UI.

Milestone / Progress flow — Entirely disconnected



Milestones are completely read-only. Engineers have no way to update actual progress %. The S-curve chart on the Dashboard uses Math.random() — it generates different fake data on every page load and has zero connection to any real milestone data.

Activity log flow — 1 of 22 routes connected



Only the GRN route creates activity log entries. All 21 other routes that change state — PO approvals, dispatches, Indent status updates, Bill generation, MB entries — produce zero activity events. The audit trail is almost entirely empty in real usage.

04 · Critical Issues & Security Gaps

The following issues must be resolved before Nexus-OP can be shown to any real client or organisation. Items marked CRITICAL represent active security vulnerabilities.



All API routes are unauthenticated

CRITICAL

Problem: Every route in `api/index.js` is publicly accessible. `PATCH /po/:id/approve`, `POST /bills/generate`, `POST /projects` — all can be called by anyone who discovers the Vercel deployment URL. The frontend role dropdown is cosmetic only.

Recommended fix: Fix: Add `supabase.auth.getUser()` middleware and apply to all routes. Reject with 401 if no valid session token is present.



server packages bundled with React frontend

CRITICAL

Problem: `express`, `cors`, and `puppeteer` are listed in dependencies alongside `React`. Vite attempts to bundle these at build time. Puppeteer downloads Chromium (~300MB) unnecessarily and causes build failures or extreme bloat.

Recommended fix: Fix: Move `express/cors` to a separate `api/package.json`. Remove `puppeteer` entirely or move it to a dedicated service worker.



Supabase anon key without Row Level Security

CRITICAL

Problem: The anon key with no RLS means any user can call the Supabase REST API directly (bypassing Express) and read or write any row in any table. Data isolation between projects relies on `?projectId=` params which are spoofable.

Recommended fix: Fix: Enable RLS on all Supabase tables. Create policies scoped to `auth.uid()`. Switch from anon key to service role key on the backend only.



Dashboard S-curve uses `Math.random()`

HIGH

Problem: The most prominent visualisation in the entire platform — the S-curve progress chart on the Dashboard — uses `Math.random()` to generate all data points. The curve changes on every page render and has no connection to milestone data.

Recommended fix: Fix: Add `actualPercent` and `plannedPercent` fields to milestones table. Drive the S-curve from real `PATCH /milestones/:id` updates by engineers.



Activity log missing for 21 of 22 routes

HIGH

Problem: The audit trail is a core value proposition of Nexus-OP. Currently only GRN receipt creates an activity entry. PO approvals, Indent updates, Bill generation, MB entries, and all 17 other state changes produce zero log events.

Recommended fix: Fix: Add `activities.insert()` calls to every state-changing route. 2 lines per route. The table structure already exists.



"Smart PDF Import" and bulk upload are fake

HIGH

Problem: The Vendors page "Smart PDF Import" is a setTimeout that pre-fills the form with "ABC Infra Pvt Ltd" regardless of what file is uploaded. The "Bulk Excel Import" button calls /api/vendors/bulk which returns 404.

Recommended fix: Fix: Implement multer for file upload handling. Use the xlsx npm package to parse Excel. Start with BOQ bulk import (highest priority).

05 · Roadmap Gap Analysis

Phase Completion vs Roadmap

The 6-phase roadmap in the product documents describes a journey from Core Platform to AI Copilot. Below is an honest assessment of current completion against each phase.



Phase 1 at 18% and Phase 2 at 22% means the platform cannot yet support a real team. The correct strategy is: complete Phase 1 before touching Phases 3–6.

PHASE 106 · Phased Implementation Plan

Phase 1 — Core Platform Stabilisation (4–6 weeks)

This is the most important phase. Goal: make Nexus-OP usable by a real organisation locally. Every task in this phase is a prerequisite for everything that follows.

1.1 · Authentication System

Implementation using Supabase Auth with JWT tokens.

```
// backend/middleware/auth.js
import supabase from "../supabase.js";

export const requireAuth = async (req, res, next) => {
  const token = req.headers.authorization?.split("Bearer ")[1];
  if (!token) return res.status(401).json({ error: "Unauthorized" });
  const { data: { user }, error } = await supabase.auth.getUser(token);
  if (error || !user) return res.status(401).json({ error: "Invalid token" });
  req.user = user;
  next();
};
```

```
// Apply to all routes in api/index.js:
Frontend Auth Flowwith); // one line protects everything
```

Create src/pages/Login.jsx with Supabase signInWithPassword. Store session in Zustand (not localStorage). Add an AuthGuard wrapper around the Router that redirects to /login if no session. Handle token refresh via supabase.auth.onAuthStateChange.

1.2 · Real RBAC Implementation

Module / Action	Admin	Engineer	Finance	Vendor	Viewer
PO Create	✓	✓	✗	✗	✗
PO Approve	✓	✗	✓	✗	✗
PO Dispatch	✓	✗	✗	✓	✗
GRN Create	✓	✓	✗	✓	✗
MB Entry	✓	✓	✗	✗	✗
Bill Generate	✓	✗	✓	✗	✗
Milestone Update	✓	✓	✗	✗	✗
Project Create	✓	✗	✗	✗	✗
Activity View	✓	✓	✓	✓	✗

1.3 · Activity Log — Add to All 21 Missing Routes

The GRN route already correctly logs to the activities table. This pattern must be applied to every state-changing route. Below is a reusable helper and the list of routes that need it.

```
// backend/services/activity.js
export const logActivity = async (supabase, { projectId, userId, type, description }) => {
  await supabase.from("activities").insert({
    projectId, userId, type, description,
    timestamp: new Date().toISOString()
  });
};

// Usage in any route:
await logActivity(supabase, {
  projectId: po.projectId,
  userId: req.user.id,
```

Route	Activity Type	Description Template
PATCH /po/:id/approve	PO_APPROVED	PO-{id} approved by {user}
PATCH /po/:id/dispatch	PO_DISPATCHED	PO-{id} dispatched to vendor
POST /po	PO_CREATED	PO created for {itemName} (qty: {qty})
POST /grn	GRN_RECEIVED	GRN-{id} received ({qty} units of {item})
PUT /indent/:id/status	INDENT_UPDATED	Indent #{id} status → {status}
POST /indent	INDENT_CREATED	Indent raised for {qty} of {item}
POST /bills/generate	BILL_GENERATED	RA Bill generated for WO-{id} (■{amt})
POST /mb	MB_ENTRY	MB entry added: {qty} {unit} at chainage {ch}
POST /projects	PROJECT_CREATED	Project "{name}" created
POST /vendors	VENDOR_ADDED	Vendor "{name}" added to project
POST /work-orders	WO_CREATED	Work Order "{name}" created

06 · Phase 1 continued — Upload Engine & Milestones

1.4 · Real Upload Engine (replacing fake stubs)

Currently both "Smart PDF Import" and "Bulk Excel Import" are setTimeout stubs. The correct implementation uses multer for file handling and the xlsx package for parsing.

```
// Install: npm install multer xlsx

// backend/routes/upload.js
import multer from "multer";
import * as XLSX from "xlsx";

const upload = multer({ storage: multer.memoryStorage() });

app.post("/upload/boq", upload.single("file"), async (req, res) => {
  const wb = XLSX.read(req.file.buffer, { type: "buffer" });
  const ws = wb.Sheets[wb.SheetNames[0]];
  const rows = XLSX.utils.sheet_to_json(ws);
  // Auto-map columns: itemCode, description, unit, estimatedQuantity, rate
  const mapped = rows.map(row => ({
    projectId: req.body.projectId,
    itemCode: row["Item Code"] || row["ItemCode"],
    description: row["Description"] || row["Description"],
    unit: row["Unit"] || row["Unit"],
    estimatedQuantity: parseFloat(row["Qty"] || row["estimatedQuantity"] || 0),
    rate: parseFloat(row["Rate"] || row["rate"] || 0),
  }));
  const { data, error } = await supabase.from("boq_items").insert(mapped).select();
  if (error) return res.status(500).json({ error: error.message });
  res.json(data[0]);
});
```

Upload types to implement (in order of priority):

- 1. BOQ Upload** — Excel with item codes, descriptions, units, quantities, rates. Highest priority — gates entire project setup.
- 2. Vendor Bulk Import** — Excel with company name, type, contact details. Replace the fake bulk upload button.
- 3. Indent Upload** — Excel with BOQ item references, quantities, required dates. Accelerates field operations.
- 4. Progress Sheet Upload** — Excel with milestone IDs and actual % values. Feeds the S-curve directly.
- 5. Inventory Upload** — Initial stock data upload for projects migrating from existing systems.

1.5 · Milestone Progress Engine

```
// New backend route:
app.patch("/milestones/:id", async (req, res) => {
  const { actualPercent, remarks } = req.body;
  const { data, error } = await supabase
    .from("milestones")
    .update({ actualPercent, remarks, updatedAt: new Date() })
    .eq("id", req.params.id)
    .select();
  if (error) return res.status(500).json({ error: error.message });
  await logActivity(supabase, {
    type: "MILESTONE_UPDATED",
    description: `Milestone #${req.params.id} updated to ${actualPercent}%`
  });
  res.json(data[0]);
});

// Add to milestones table via Supabase migration:
// ALTER TABLE milestones ADD COLUMN actualPercent NUMERIC DEFAULT 0;
// ALTER TABLE milestones ADD COLUMN plannedPercent NUMERIC DEFAULT 0;
// ALTER TABLE milestones ADD COLUMN remarks TEXT;
```

PHASE 2

06 · Phase 2 — Workflow & Execution Engine

Phase 2 — Workflow & Execution Engine (6–10 weeks)

2.1 · Bill Approval Lifecycle

Bills currently stop at "Draft" with no forward progression. Add a proper state machine: Draft → Submitted → Finance Approved → Paid.

```
// New routes to add:
app.patch("/bills/:id/submit", ...) // Draft → Submitted
app.patch("/bills/:id/approve", ...) // Submitted → Finance Approved (Finance role only)
app.patch("/bills/:id/pay", ...) // Finance Approved → Paid (Admin only)
app.patch("/bills/:id/reject", ...) // Any → Rejected (with reason)
```

2.2 · Connect Indent → PO Auto-Creation

When an Indent is approved, the system should offer to create a PO automatically. The indentId field already exists in purchase_orders — it just needs to be wired.

```
// In PUT /indent/:id/status when status = "Approved":
if (status === "Approved") {
  const indent = await supabase.from("indents").select("*").eq("id", id).single();
  // Emit an event for the frontend to show "Create PO from this indent?" prompt
  res.json({ success: true, suggestPO: true, indentData: indent.data });
}
```

2.3 · Dynamic Process Flow (Live Data)

The ProcessFlow page currently has hardcoded nodes. It should auto-generate a flow diagram from the actual PO/GRN/Indent state for the active project.

2.4 · Role-Specific Dynamic Dashboards

Admin Dashboard: Overall project health score, Financial exposure (total PO value vs billed), Risk projects (delayed milestones), Vendor performance overview

Engineer Dashboard: Pending milestone updates, Assigned work orders, Pending indents awaiting approval, Recent MB entries

Finance Dashboard: Bills pending approval, Monthly cash flow, TDS/retention totals, Outstanding PO values

Vendor Dashboard: My assigned POs, Dispatch status tracker, Invoice/bill status, Delivery schedule

PHASE 3

06 · Phases 3–6 Overview

Phases 3–6 — Future Roadmap (Months 4–18+)

Phase 3 · Realtime + Geo Intelligence

Timeline: 2–4 months after Phase 2

- Supabase Realtime subscriptions for live PO/GRN status updates across users
- Replace hardcoded ORRMap with dynamic markers from the DB (lat/lng on projects table)
- Geo-tagged uploads: every evidence upload stores coordinates + timestamp + userId
- PostGIS extension for geofencing and spatial queries
- Live site monitoring map: project zones, material movement, issue markers

Phase 4 · Analytics & Engineering Intelligence

Timeline: 3–5 months after Phase 3

- KPI Engine: SPI (Schedule Performance Index), CPI (Cost Performance Index), burn rate
- Forecasting: delay probability, material shortage predictions, cost overrun alerts
- Vendor analytics: delivery performance scores, delay frequency, quality metrics
- Project health score: composite risk score with execution confidence rating
- Inventory intelligence: projected depletion dates, reorder triggers, wastage trends

Phase 5 · Visual Progress Intelligence

Timeline: 4–6 months after Phase 4

- Engineer photo uploads with GPS geotag, timestamp, and project linkage
- Before/after visual timeline per milestone and site location
- AI image analysis for construction stage detection (Claude Vision API)
- Visual evidence attached to MB entries for billing verification

Phase 6 · AI Operational Copilot

Timeline: Long-term (12–18 months)

- Prompt-based planning: "Build a bridge" → auto-generates BOQ, milestones, workflows
- AI assistant: "Why is this delayed?" → analyses activity log + milestone data
- AI recommendations: procurement optimisation, manpower redistribution suggestions
- Simulation engine: model delays, cost spikes, material shortages

07 · Priority Action Plan — What to Build Next

This is the ordered list of work items to execute immediately. Do not start any Phase 2 work until items 1–3 are complete. Do not build AI features until item 5 is complete.

1

Supabase Auth + Middleware Guard on All Routes

Estimated: 3–5 days

CRITICAL

- Create backend/middleware/auth.js with requireAuth function
- Add app.use(requireAuth) before all routes in api/index.js
- Create src/pages/Login.jsx with email/password form
- Add supabase.auth.onAuthStateChange listener in main.jsx
- Create AuthGuard component wrapping Router — redirect to /login if no session
- Update Sidebar to show logged-in user name + logout button
- Add Authorization: Bearer {token} header to all axios/fetch calls

2

Activity Log on All 21 Missing Routes

Estimated: 1–2 days

CRITICAL

- Create backend/services/activity.js with logActivity helper
- Add logActivity call to: PATCH /po/:id/approve, PATCH /po/:id/dispatch
- Add logActivity call to: POST /po, POST /grn, POST /mb, POST /indent
- Add logActivity call to: PUT /indent/:id/status, POST /bills/generate
- Add logActivity call to: POST /projects, POST /vendors, POST /work-orders
- Include userId from req.user in every log entry
- Update ActivityLog.jsx to show user, module, type, and description

3

Fix Package.json — Separate Server from Frontend

Estimated: 0.5 days

CRITICAL

- Create api/package.json with express, cors as dependencies
- Remove express, cors, puppeteer from root package.json
- Update vercel.json builds config to install api/package.json separately
- Verify Vite build completes without server module errors

4

Enable Supabase RLS on All Tables

CRITICAL

Estimated: 1–2 days

- Enable RLS: ALTER TABLE projects ENABLE ROW LEVEL SECURITY
- Create policies for each table scoped to auth.uid()
- Switch backend to use SUPABASE_SERVICE_ROLE_KEY (bypasses RLS server-side)
- Keep SUPABASE_ANON_KEY only for client-side auth token exchange
- Test: verify direct Supabase REST calls without auth token are rejected

5

Real Milestone Progress + S-Curve from Live Data

HIGH

Estimated: 2–3 days

- Supabase migration: add actualPercent, plannedPercent, remarks to milestones
- Add PATCH /milestones/:id backend route
- Update Milestones.jsx with progress input slider per milestone
- Replace Math.random() in Dashboard.jsx with real milestone data fetch
- S-curve: planned line from plannedPercent values, actual from actualPercent values

6

Real Excel Upload Engine (BOQ first)

HIGH

Estimated: 3–4 days

- Install: npm install multer xlsx (in api/package.json)
- Create POST /upload/boq route with multer.single("file")
- Parse Excel with XLSX.utils.sheet_to_json, auto-map column names
- Add preview endpoint that returns first 5 rows before committing
- Update BOQ.jsx with drag-and-drop upload zone and column mapping UI
- Repeat pattern for Vendors bulk import (lower priority)

7

Single Axios API Client + Zustand Store

HIGH

Estimated: 2–3 days

- Create src/services/api.js — axios instance with baseURL and auth interceptor
- Add request interceptor to attach Authorization: Bearer {token} automatically
- Add response interceptor to handle 401 (redirect to login) globally
- Install Zustand: npm install zustand
- Create src/stores/useProjectStore.js replacing ProjectContext
- Create src/stores/useAuthStore.js for session + user state
- Refactor all pages to use api.get/api.post instead of direct axios/fetch

8

Bill Approval Lifecycle

MEDIUM

Estimated: 2–3 days

- Add routes: PATCH /bills/:id/submit, /approve, /pay, /reject
- Update bills table: add approvedBy, paidAt, rejectionReason columns
- Update Bills.jsx with action buttons per bill status
- Restrict approve to Finance role, pay to Admin role (check req.user.role)

08 · File Change Tracker

Complete list of every file that needs to be created or modified, organised by phase and priority. Use this as your development checklist.

New Files to Create

Priority 1 — Auth & Security

File Path	Purpose
api/middleware/auth.js	requireAuth middleware — validates Supabase JWT token on every request
api/middleware/rbac.js	requireRole(roles[]) middleware — checks user role against allowed list
api/services/activity.js	logActivity(supabase, {projectId, userId, type, description}) helper
src/pages/Login.jsx	Email + password login form using supabase.auth.signInWithPassword
src/pages/Register.jsx	Invite-based registration (optional, can use Supabase dashboard initially)
src/components/AuthGuard.jsx	Wraps Router — redirects to /login if no active Supabase session
src/stores/useAuthStore.js	Zustand store: session, user, login(), logout() actions
src/stores/useProjectStore.js	Zustand store replacing ProjectContext, adds loading/error states
src/services/api.js	Central axios instance with auth interceptor + 401 handler

Priority 2 — Upload Engine

File Path	Purpose
api/routes/upload.js	multer + xlsx routes: POST /upload/boq, /upload/vendors, /upload/indent
src/components/UploadZone.jsx	Drag-and-drop file upload with preview, validation, column mapping UI
src/components/ColumnMapper.jsx	Maps detected Excel columns to expected schema fields

Priority 3 — Workflow & State

File Path	Purpose
api/routes/bills.js	Bill approval routes: submit, approve, pay, reject with role guards
api/routes/milestones.js	PATCH /milestones/:id for progress updates + remarks
src/components/WorkflowStepper.jsx	Visual stepper showing current state in any workflow (PO, Bill, Indent)
src/components/ApprovalModal.jsx	Confirmation modal for approval actions with reason/notes input
src/hooks/useRealtime.js	Supabase realtime subscription hook for live table updates

Files to Modify (Significant Changes)

File	What Changes	Impact
api/index.js	Add app.use(requireAuth). Add logActivity to 21 routes. Extract routes to separate files.	CRITICAL
api/supabase.js	Add service role key for backend. Keep anon key for auth only.	CRITICAL
package.json	Remove express/cors/puppeteer. Keep only React/Vite/frontend deps.	CRITICAL
src/App.jsx	Wrap routes in AuthGuard. Remove TopHeader role dropdown (replace with real user).	HIGH
src/context/RoleContext.jsx	Replace useState("Admin") with role from JWT claims or users table.	HIGH

src/context/ProjectContext.jsx	Migrate to Zustand useProjectStore. Add loading/error states.	HIGH
src/pages/Dashboard.jsx	Replace Math.random() S-curve with real milestone data API call.	HIGH
src/pages/Milestones.jsx	Add progress update UI, PATCH API calls, project-scoped filter.	HIGH
src/pages/Vendors.jsx	Replace setTimeout fake upload with real UploadZone component.	HIGH
src/pages/Bills.jsx	Add submit/approve/pay/reject action buttons per bill status.	HIGH
src/pages/ActivityLog.jsx	Add user name, activity type badge, timestamp formatting.	MEDIUM
src/pages/Inventory.jsx	Add manual adjustment, consumption tracking, depletion indicators.	MEDIUM
src/pages/ProcessFlow.jsx	Drive nodes from live project PO/GRN/Indent data instead of hardcoded.	MEDIUM
src/components/ORRMap.jsx	Replace hardcoded coordinates with dynamic markers from projects table.	MEDIUM
vercel.json	Update build config to handle separate api/package.json installation.	LOW

Database Schema Changes Required

```
-- Run these migrations in Supabase SQL editor:

-- 1. Milestones: progress tracking
ALTER TABLE milestones ADD COLUMN actualPercent NUMERIC DEFAULT 0;
ALTER TABLE milestones ADD COLUMN plannedPercent NUMERIC DEFAULT 0;
ALTER TABLE milestones ADD COLUMN remarks TEXT;
ALTER TABLE milestones ADD COLUMN updatedAt TIMESTAMPTZ;

-- 2. Bills: approval lifecycle
ALTER TABLE bills ADD COLUMN approvedBy UUID REFERENCES auth.users(id);
ALTER TABLE bills ADD COLUMN approvedAt TIMESTAMPTZ;
ALTER TABLE bills ADD COLUMN paidAt TIMESTAMPTZ;
ALTER TABLE bills ADD COLUMN rejectionReason TEXT;

-- 3. All tables: soft deletes + audit timestamps
ALTER TABLE projects ADD COLUMN deletedAt TIMESTAMPTZ;
ALTER TABLE purchase_orders ADD COLUMN createdBy UUID REFERENCES auth.users(id);
ALTER TABLE purchase_orders ADD COLUMN approvedBy UUID REFERENCES auth.users(id);

-- 4. Projects: geo coordinates for map
ALTER TABLE projects ADD COLUMN latitude NUMERIC;
ALTER TABLE projects ADD COLUMN longitude NUMERIC;

-- 5. Activities: add userId and type
ALTER TABLE activities ADD COLUMN userId UUID REFERENCES auth.users(id);
ALTER TABLE activities ADD COLUMN type VARCHAR(50);

-- 6. Enable Row Level Security on all tables:
ALTER TABLE projects ENABLE ROW LEVEL SECURITY;
ALTER TABLE purchase_orders ENABLE ROW LEVEL SECURITY;
ALTER TABLE milestones ENABLE ROW LEVEL SECURITY;
-- (repeat for all tables)
```

09 · Architecture Refactor Guide

Target Backend Structure

```

backend/
  ■■■ index.js           ← Express app setup, middleware registration only
  ■■■ supabase.js        ← Service role client (server use only)
  ■■■ middleware/
  ■   ■■■ auth.js        ← requireAuth: JWT validation
  ■   ■■■ rbac.js        ← requireRole(["Admin","Finance"])
  ■   ■■■ errorHandler.js ← Global error middleware
  ■■■ routes/
  ■   ■■■ projects.js    ← GET/POST /projects
  ■   ■■■ vendors.js     ← GET/POST /vendors + bulk upload
  ■   ■■■ po.js          ← GET/POST /po + approve/dispatch
  ■   ■■■ grn.js         ← POST /grn
  ■   ■■■ inventory.js   ← GET /inventory + adjustments
  ■   ■■■ bills.js       ← POST/PATCH /bills + approval flow
  ■   ■■■ milestones.js  ← GET/POST/PATCH /milestones
  ■   ■■■ boq.js         ← GET/POST /boq
  ■   ■■■ indent.js      ← GET/POST/PUT /indent
  ■   ■■■ mb.js          ← GET/POST /mb
  ■   ■■■ workorders.js  ← GET/POST /work-orders
  ■   ■■■ upload.js      ← POST /upload/* (multer routes)
  ■   ■■■ activities.js  ← GET /activities
  ■   ■■■ dashboard.js   ← GET /dashboard (aggregation)
  ■■■ services/

```

Target Frontend Structure

```

src/
  ■■■ App.jsx           ← Router only, AuthGuard wrapper
  ■■■ main.jsx          ← Supabase auth listener init
  ■■■ services/
  ■   ■■■ api.js        ← Axios instance: base URL + auth interceptor
  ■■■ stores/
  ■   ■■■ useAuthStore.js ← Zustand: session, user, role
  ■   ■■■ useProjectStore.js ← Zustand: projects, activeProject
  ■   ■■■ useUIStore.js  ← Zustand: sidebar, modals, notifications
  ■■■ hooks/
  ■   ■■■ usePO.js       ← fetch POs, approve, dispatch actions
  ■   ■■■ useMilestones.js ← fetch + update milestones
  ■   ■■■ useVendors.js  ← fetch, add, bulk import
  ■   ■■■ useRealtime.js  ← Supabase realtime subscriptions
  ■■■ modules/
  ■   ■■■ procurement/   ← PO, GRN, Indent, Vendors pages
  ■   ■■■ execution/     ← Milestones, MeasurementBook, BOQ
  ■   ■■■ finance/       ← Bills, RA billing
  ■   ■■■ projects/      ← Projects, WorkOrders
  ■   ■■■ analytics/     ← Dashboard, ActivityLog, ProcessFlow
  ■■■ shared/
  ■   ■■■ DataTable.jsx  ← Reusable sortable/filterable table
  ■   ■■■ StatusBadge.jsx ← Consistent status pill component
  ■   ■■■ UploadZone.jsx  ← Drag-and-drop with progress
  ■   ■■■ ConfirmModal.jsx ← Approval/destructive action confirmation
  ■   ■■■ ActivityFeed.jsx ← Reusable activity timeline
  ■   ■■■ WorkflowStepper.jsx ← Visual state machine progress indicator
  ■■■ layouts/
  ■   ■■■ AppLayout.jsx  ← Sidebar + main content shell
  ■   ■■■ AuthLayout.jsx ← Centered card for Login/Register

```

10 · Success Metrics & Completion Criteria

Use these criteria to verify each phase is truly complete before proceeding. "It works on my machine" is not a completion criterion.

Phase 1 Completion Criteria

Auth

- Un-authenticated request to /api/po returns 401
- Login form works and persists session across page refresh
- Logout clears session and redirects to /login
- JWT token is sent in Authorization header on all API calls

RBAC

- Finance role cannot call /api/po (approve) — returns 403
- Vendor role cannot see BOQ, MB, or Milestones pages
- Admin can access all routes without restriction

Activity Log

- After approving a PO, an entry appears in the Activity Log
- Activity log shows user name, not "unknown"
- All 21 routes create entries — verify with 10-minute usage session

Uploads

- BOQ Excel file with 50 rows imports successfully
- Column mapping UI shows preview before commit
- Invalid file type shows an error, not a crash

S-Curve

- S-curve does not change on page refresh (no Math.random)
- After engineer updates milestone to 60%, S-curve reflects it
- Planned and actual lines are visually distinct

Phase 2 Completion Criteria

Bills

- Bill cannot be approved by an Engineer role — returns 403
- After Finance approves a bill, status changes to "Finance Approved"
- Bill approval creates an activity log entry

Workflow

- Approving an Indent offers a "Create PO" prompt with pre-filled data
- Process Flow diagram updates when a new PO is created
- Role-specific dashboard shows different data for Admin vs Finance

The North Star Metric

Before any Phase 2 work: one real organisation must be using Nexus-OP daily for at least two weeks with real project data. That validation loop — real usage, real friction, real workflow refinement — is worth more than any AI feature or geo intelligence system. Build the foundation right first.

Phase	Timeline	Current	Key Deliverable
Phase 1 — Core	4–6 weeks	18%	Auth, Activity Log, Uploads, Milestones
Phase 2 — Workflow	6–10 weeks	22%	Bill lifecycle, Indent→PO, Role dashboards
Phase 3 — Realtime+Geo	2–4 months	8%	Live updates, Dynamic maps, Geo tags
Phase 4 — Analytics	3–5 months	5%	KPIs, Forecasting, Health scores
Phase 5 — Visual	4–6 months	0%	Image tracking, Site timeline
Phase 6 — AI Copilot	12–18 months	0%	Prompt planning, AI assistant

NEXUS-OP · Operational Intelligence Platform for Infrastructure Execution

"Every real-world operational activity should become digitally observable and intelligently connected."